Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Systems@ETH Zürich

Cloud Computing Architecture

Semester project report

Group 54
Aaron Achermann - 20-940-805
Indro Born - 21-936-208
Matteo Pelossi – 22-952-444

Systems Group
Department of Computer Science
ETH Zurich
May 15, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

  Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.

- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.

- **Use of AI Tools**: The use of AI tools must adhere to ETH regulations. **You take responsibility for the content you submit**. You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 3 [68 points]

1. **[34 points]** Describe and analyze your hand-crafted policy.

   (a) **[17 points]** With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time [1] across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency $>$ 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

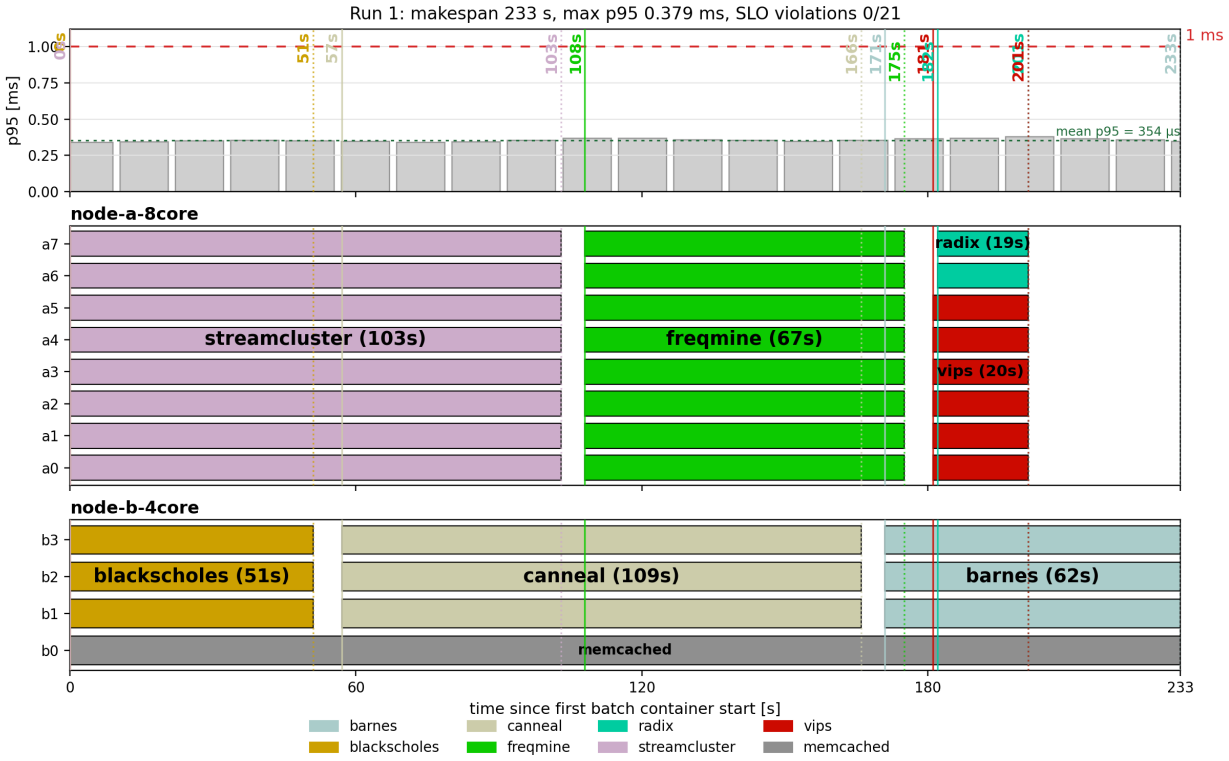
job name	mean time [s]	std [s]
barnes	63.33	1.53
blackscholes	47.67	3.06
canneal	107.33	1.53
freqmine	67.33	0.58
radix	20.67	2.08
streamcluster	113.33	16.20
vips	21.33	1.53
total time	232.33	3.06

Answer: The values in the table are computed from the container `startedAt` and `finishedAt` timestamps in `results.json`. In the augmented `mcperf` output, `ts_start` and `ts_end` are Unix epoch timestamps in milliseconds that bound the measurement interval represented by one p95 latency row. All three executions completed the seven batch jobs successfully. During the interval from the first batch container start to the last batch container finish, no measured memcached p95 sample exceeded the 1 ms SLO; the resulting SLO violation ratio is 0.

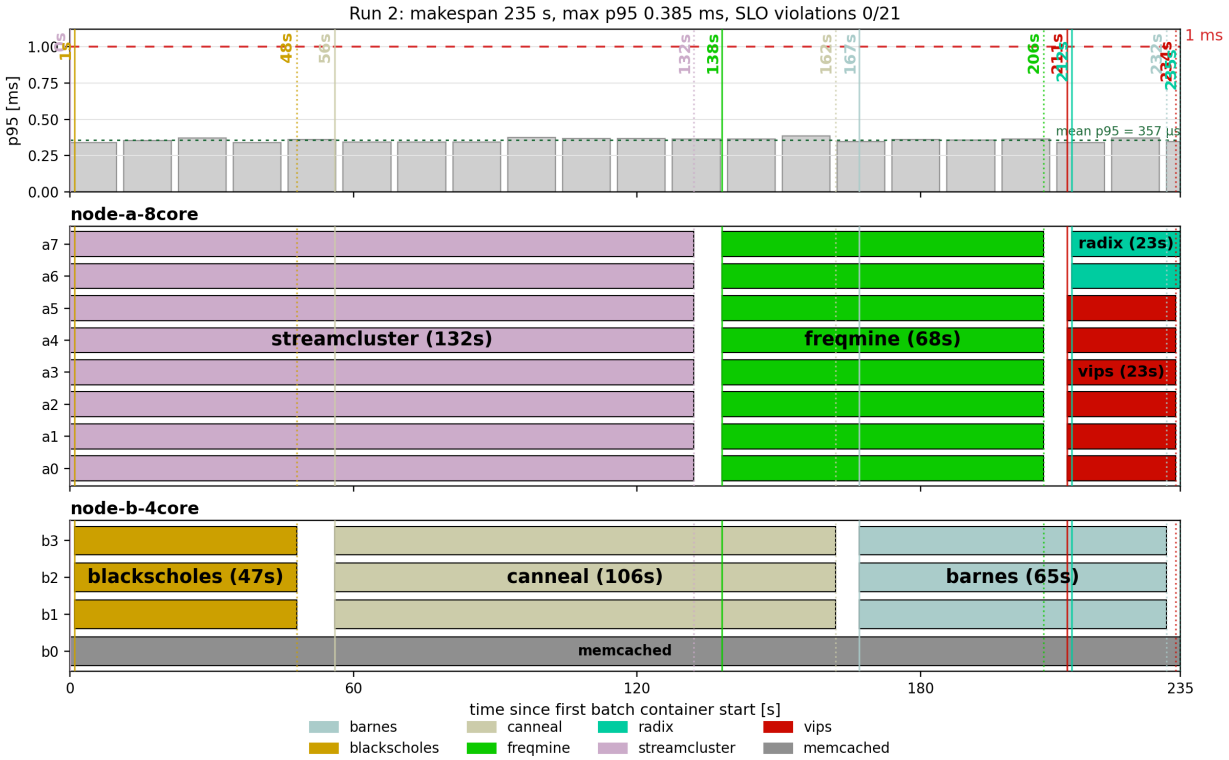
Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in the color of vips for core x from when vips is started to when it ends if vips ran on core x.

Plots:

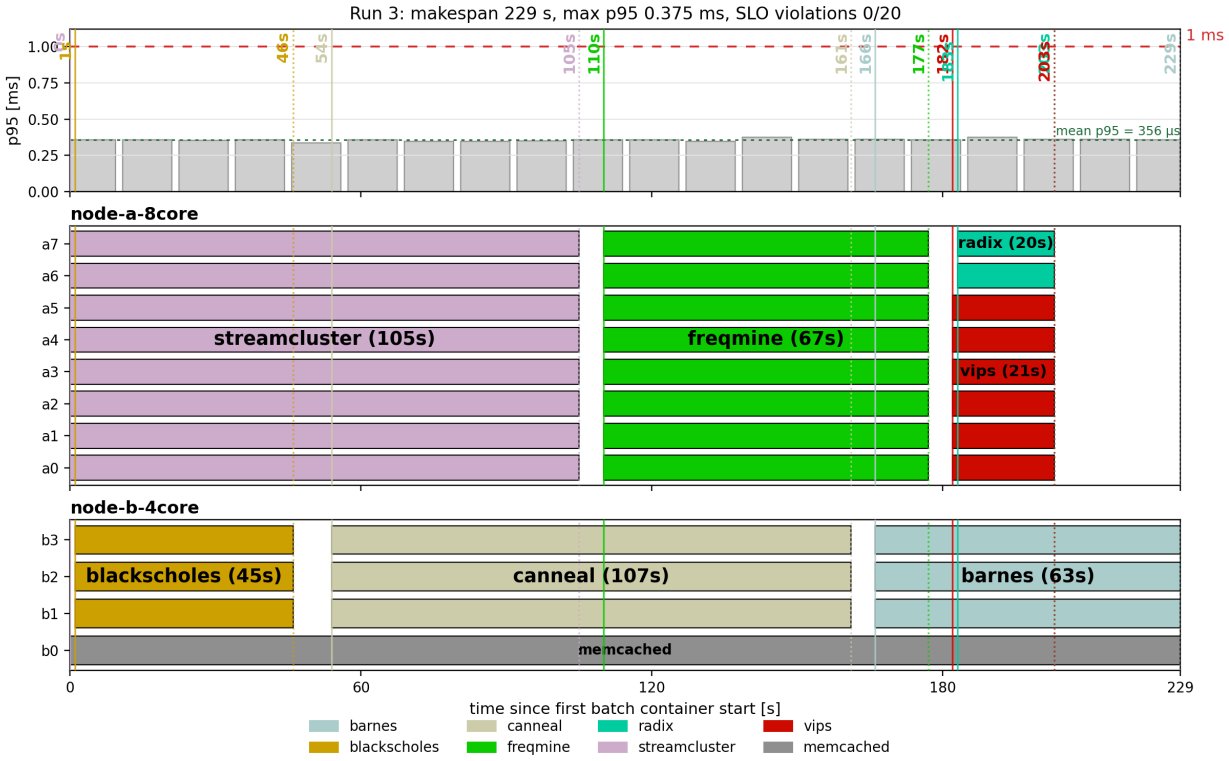
[1] Here, you should only consider the runtime, excluding time spans during which the container is paused.



Run 1. Memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.



Run 2. Memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.



Run 3. Memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.

(b) **[17 points]** Describe and justify the "optimal" scheduling policy you have designed.

- Which node does memcached run on? Why?
  Answer: Memcached runs on `node-b-4core`, pinned to core `0` with one server thread. The reasoning behind the placement of the memcached pod is multifaceted. The first constraint that immediately came to our attention was the physical shape of the two benchmark machines. By analyzing `part3.yaml`, we noticed that the two VMs request different resources and have different shapes: `node-a-8core` is an `e2-standard-8` machine, while `node-b-4core` is an `n2d-highcpu-4` machine. The `e2` shape has a less predictable CPU allocation: it can map to slower Intel CPUs or to a faster AMD Rome CPU. In our captured logs, however, the 8-core VM was consistently assigned AMD Rome. The second node, `node-b-4core`, was an `n2d` high-CPU machine and in our experiments was consistently assigned AMD Milan. The most important difference is therefore not only the number of cores, but also the memory density and CPU shape: `node-a-8core` has 8 cores and about 32 GB of memory, while `node-b-4core` has only 4 cores and about 4 GB of memory.
  Given this information, we tried to build the policy using only the knowledge extracted from Part 1 and Part 2. From Part 1, the immediate observation was that memcached was less susceptible to DRAM bandwidth interference than to CPU and cache interference. This made it reasonable to keep memcached on a single isolated core, while allowing the remaining three cores of the same machine to execute batch jobs sequentially. We did not want memcached to compete directly for a core, for

L1d/L1i, or for L2 with a batch job. Pinning it to core `0` on the AMD Milan node gave us a stronger and more predictable latency-critical core, and cores `1-3` remained available for the node-B job sequence. Across the three selected final-policy runs, this placement produced zero memcached SLO violations.

- Which node does each of the 7 batch jobs run on? Why?
  Answer:
  - barnes: `barnes` runs on `node-b-4core`, pinned to cores `1-3`, with 3 threads. We place it at the end of the node-B chain, after `canneal`. Although `barnes` benefits from more cores, it is also cache-sensitive, especially to LLC pressure. At this point in the schedule, `node-a-8core` is already used by the final `vips`+`radix` overlap, while `node-b-4core` has three isolated batch cores available next to the memcached core. Running `barnes` there avoids adding a third job to node A and keeps the policy simple and reproducible.
  - blackscholes: `blackscholes` runs on `node-b-4core`, pinned to cores `1-3`, with 3 threads. This is one of the safest jobs to place next to memcached: from Part 2 it showed only moderate interference sensitivity, and from the speedup experiment it did not gain much from going from 4 to 8 threads. Therefore it is a good first job for the 3-core node-B batch region, while memcached remains isolated on core `0`.
  - canneal: `canneal` runs on `node-b-4core`, pinned to cores `1-3`, with 3 threads, after `blackscholes`. `canneal` is memory and LLC sensitive, but it also has poor scalability compared with the jobs that benefit most from all 8 cores. Since the second machine has only three batch cores once memcached is allocated, we use it for jobs where the opportunity cost of not using 8 cores is lower. The measured runs confirmed that this placement stayed within the memcached SLO.
  - freqmine: `freqmine` runs on `node-a-8core`, pinned to cores `0-7`, with 8 threads, after `streamcluster`. From Part 2, `freqmine` has a significant speedup at high thread counts, especially from 4 to 8 threads. It also has irregular control flow and cache pressure, so we preferred not to colocate it with memcached. Giving it the full 8-core node reduces the critical path without risking direct interference on memcached's private caches.
  - radix: `radix` runs on `node-a-8core`, pinned to cores `6-7`, with 2 threads, after `freqmine`. During experimentation we discovered that `radix` was a highly unpredictable and unstable job. We therefore never schedule it on `node-b-4core`, because it can crash the 4-core, 4 GB VM. We also restrict its thread count to a power-of-two value from $\{1, 2, 4, 8\}$, because non-power-of-two thread counts can make it crash. The final schedule uses 2 threads, which satisfies this constraint and lets `radix` share the tail of `node-a-8core` with `vips` on disjoint cores.
  - streamcluster: `streamcluster` runs on `node-a-8core`, pinned to cores `0-7`, with 8 threads, and starts immediately. This job showed the strongest scalability in Part 2, so the 8-core node is the natural place for it. Starting it first also makes sense because it is one of the longest jobs in the workflow; delaying it would directly increase the makespan.
  - vips: `vips` runs on `node-a-8core`, pinned to cores `0-5`, with 6 threads, after `freqmine`. `vips` benefits from parallelism but has diminishing returns at high thread counts, so six cores are enough to make it short while leaving cores `6-7`

for `radix`. This is the only intentional batch-job colocation in the final tail of the schedule, and the two jobs use disjoint core sets.

- Which jobs run concurrently / are colocated? Why?
  Answer: We use "concurrent" to mean that two jobs overlap in time, and "colocated" to mean that they overlap on the same VM. Concurrency across different VMs is mainly a makespan choice: it does not create direct CPU-cache contention between the two batch jobs, but it keeps both paid benchmark nodes busy. Colocation is the more delicate decision, because colocated jobs can compete for cores, caches, memory bandwidth, and memory capacity.
  With this distinction, the final policy runs the node-A and node-B work streams at the same time only to avoid leaving a machine idle. On `node-a-8core`, streamcluster runs first on the full 8-core machine, followed by freqmine on the full 8-core machine. On `node-b-4core`, blackscholes, canneal, and barnes run sequentially on cores `1-3`, while memcached stays isolated on core `0`.
  The only intentional same-node batch colocation is at the tail: vips and radix both start after freqmine on `node-a-8core`. vips uses cores `0-5`, while radix uses cores `6-7`. We chose this colocation because both jobs are short tail jobs and, in our empirical experimentation, running them one after the other gave little end-to-end benefit once the roughly 6 s container startup overhead was included. Keeping radix on node A also avoids the instability we observed when trying to run it on the 4-core, 4 GB node. Thus, the policy allows overlap when it reduces idle time, but avoids same-core competition and uses same-node colocation only once, with disjoint core sets.

- In which order did you run the 7 batch jobs? Why?
  Order: streamcluster and blackscholes start first. Then freqmine starts after streamcluster, while canneal starts after blackscholes. Finally, vips and radix start together after freqmine, and barnes starts after canneal.
  Why: The order is chosen to keep both machines busy while respecting the different strengths of the two nodes. The higher core count of `node-a-8core` is used first for the jobs with the clearest 8-thread benefit: streamcluster and then freqmine. On the other side, after allocating one core to memcached, `node-b-4core` only has three batch cores available, so it runs a sequential chain of 3-thread jobs: blackscholes, canneal, and barnes. The tail of the schedule is then compressed by running vips and radix concurrently on disjoint node-A cores. This ordering was a direct consequence of the Part 2 speedup results and of the practical observation that container startup time becomes important for short jobs.

- How many threads have you used for each of the 7 batch jobs? Why?
  Answer:
  - barnes: 3 threads, pinned to cores `1-3` on `node-b-4core`. This matches the available batch cores on node B while keeping memcached isolated on core `0`.
  - blackscholes: 3 threads, pinned to cores `1-3` on `node-b-4core`. This job does not gain enough from 8 threads to justify using node A, so three threads are the natural choice on the memcached node.
  - canneal: 3 threads, pinned to cores `1-3` on `node-b-4core`. Its scalability is

limited by memory/cache behavior, so we avoid spending the full 8-core machine on it in the final policy.

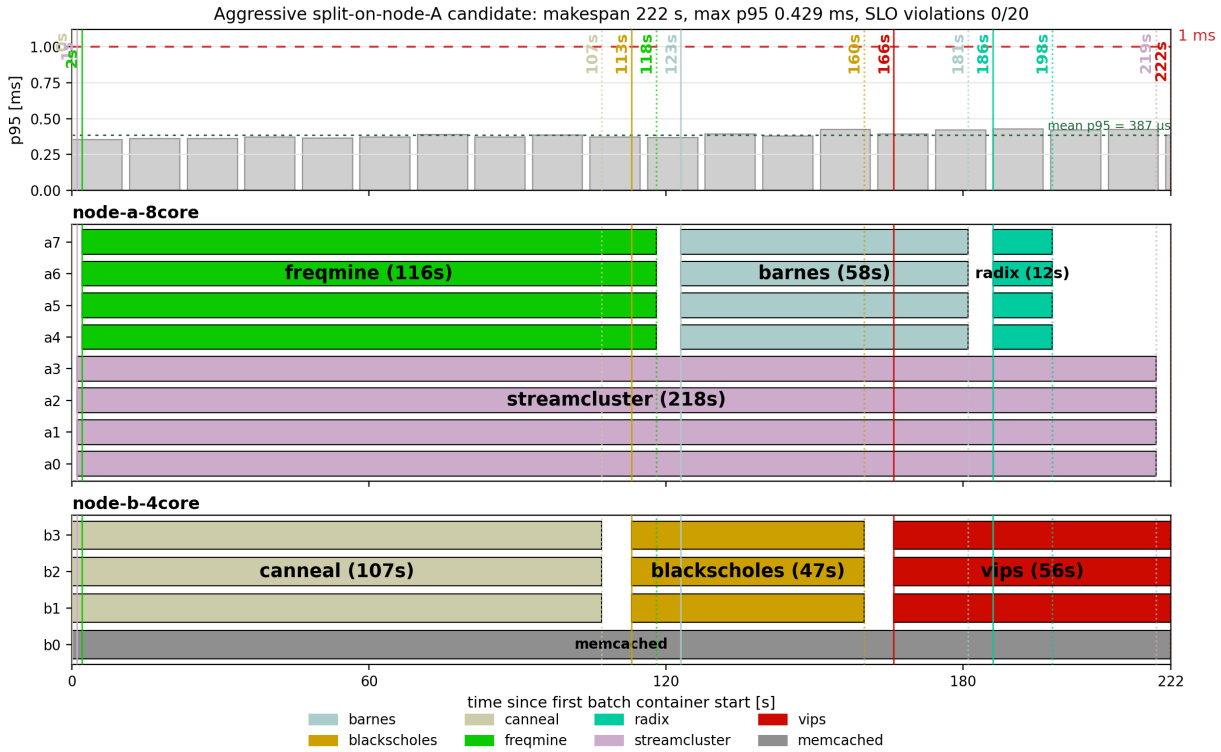
- freqmine: 8 threads, pinned to cores `0-7` on `node-a-8core`. The Part 2 speedup experiment showed that it benefits significantly from the jump to 8 threads.
- radix: 2 threads, pinned to cores `6-7` on `node-a-8core`. This choice is not only a performance choice, but also a stability constraint: `radix` must use one of $\{1, 2, 4, 8\}$ threads and should not be scheduled on `node-b-4core`. Two threads are enough for the tail overlap with `vips`, while staying inside the safe power-of-two thread set.
- streamcluster: 8 threads, pinned to cores `0-7` on `node-a-8core`. This was the strongest scaling workload in Part 2, so it receives the full 8-core node.
- vips: 6 threads, pinned to cores `0-5` on `node-a-8core`. Six threads preserve most of the parallel benefit while leaving two disjoint cores for `radix`.

- Which files did you modify or add and in what way? Which Kubernetes features did you use?
  Answer: Besides the final policy YAML files, the main addition was a Python automation layer for the whole Part 3 workflow. We built it because, before this, the workflow was mostly manual: creating or validating the cluster, entering VMs by SSH, finding IP addresses, checking whether the client machines had finished bootstrapping, building the client-side `mcperf` tooling, running every `kubectl` command, and collecting the outputs all had to be done by hand. The automation therefore became the coordination script for the experiment.
  We also modified `part3.yaml`. This file defines the kOps instance groups for the benchmark machines, including the `e2-standard-8` `node-a-8core` machine and the `n2d-highcpu-4` `node-b-4core` machine. It also contains the kOps `additional UserData` bootstrap scripts for the client machines, which install the needed build dependencies, build the augmented `mcperf` tool, and start the long-lived load-agent services. The actual stable naming of the machines is then handled by the Python code: since kOps creates Kubernetes node names with a random suffix, the automation discovers the real nodes, maps them back to logical aliases such as `node-a-8core` and `node-b-4core`, and applies or repairs the canonical project node-type labels before running a policy.
  The scheduling decisions themselves were kept in YAML policy files. This made the policy declarative: for each job the file states the target node, the pinned core range, the thread count, and the `after` dependency. The Python controller validates the policy before spending time on a run, especially checking that thread counts do not exceed the number of pinned cores. It then acts as an asynchronous DAG scheduler: it launches all jobs in a phase that is ready, keeps polling Kubernetes for the state of the already launched Jobs, records when their completion conditions are satisfied, and only then starts the dependent phases. This let us express both parallel starts and sequential dependencies without manually watching pods and launching the next command by hand.
  Around this scheduler we also automated cluster deployment and validation, SSH/IP discovery for the clients, container image pre-caching before the benchmark suite, policy queues for trying multiple schedules, result collection, log collection, and summary generation. Each run stores the rendered Kubernetes manifests, the pol-

icy snapshot, the phase plan, the raw pod snapshot, the `mcperf` output, logs, and a summarized result. This was a major quality-of-life improvement for reproducibility, because a run could be repeated from the same policy file instead of being reconstructed from a sequence of manual SSH and `kubectl` commands.
We also added validation and visualization tools around this workflow. The audit code contains a rudimentary resource-contention checker: it uses the Part 2 speedup/runtime measurements and the running statistics collected from previous runs to warn about suspicious overlaps, core collisions, and missing runtime estimates. The same running statistics are used by the web interface for creating new scheduling policies and by the run viewer for visualizing past executions as horizontal timelines; screenshots of both views are included in the appendix.
The Kubernetes mechanisms themselves remain simple: `nodeSelector` binds memcached and each batch job to the intended logical node type; Kubernetes Pods run memcached and the image pre-cache steps; Kubernetes Jobs run the PARSEC and SPLASH-2x batch containers; and managed labels make cleanup and collection reproducible. Kubernetes decides VM placement through labels and selectors, while the actual per-core isolation is done inside the container command with `taskset`.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).
  Answer: The most important design choice was to keep the policy simple and constrained. We decided not to explore simultaneous multithreading or hyperthreading policies: we did not test schedules where two jobs compete on the same core, and we did not use a thread count larger than the number of pinned cores. This reduced the search space and matched the lessons from Part 1 and Part 2. Memcached was much more vulnerable to CPU and cache interference than to pure DRAM bandwidth pressure, and the batch jobs also showed strong sensitivity to L1d, L1i, L2, and LLC effects. Therefore, whenever two jobs overlap, the selected final policy keeps their core sets disjoint.
  Another trade-off was between giving the full 8-core node to every scalable job and reducing the overall makespan. Jobs such as streamcluster and freqmine deserved the full `node-a-8core` machine because their 4-to-8-thread speedup was significant and their runtime was long enough to affect the critical path. For shorter tail jobs, however, the container startup time became visible in the end-to-end time. We measured this overhead at about 6 s when the image was already cached. This changed the tail decision: running vips and radix concurrently on disjoint node-A cores was better than launching both sequentially just to give each one more cores.
  The closest competing policy we designed was the aggressive split-on-node-A candidate. It kept memcached on `node-b-4core` but split `node-a-8core` into two 4-core regions: streamcluster ran on one half, while freqmine, barnes, and radix used the other half in sequence. On one occasion this policy produced the best observed makespan, 222 s, with zero SLO violations. However, this result was not as reproducible as the final policy. In later repetitions the same aggressive design was closer to 235–236 s, and the long streamcluster run overlapped with several other jobs on the same VM, making the runtime more dependent on shared cache state and on

the cloud platform conditions at that time of day.



Best observed execution of the aggressive split-on-node-A candidate. This run reached 222 s with zero SLO violations, but later repetitions of the same idea were slower and less stable.

We therefore selected the more conservative final hand-crafted policy. It gives the full node A to streamcluster and then to freqmine, moves barnes to the three batch cores on node B, and only overlaps vips with radix at the tail on disjoint node-A cores. It also keeps radix away from the unstable node-B placement and uses a safe power-of-two thread count. The selected final policy was slightly less aggressive than the best single observed run of the competing policy, but across the three selected executions it had a mean makespan of 232.33 s, a standard deviation of 3.06 s, and zero memcached SLO violations. We chose it because it was fast, easy to reason about, and more stable across repeated runs.

2. **[34 points]** Describe and analyze the AI-generated policy.

   (a) **[17 points] Report the performance**:

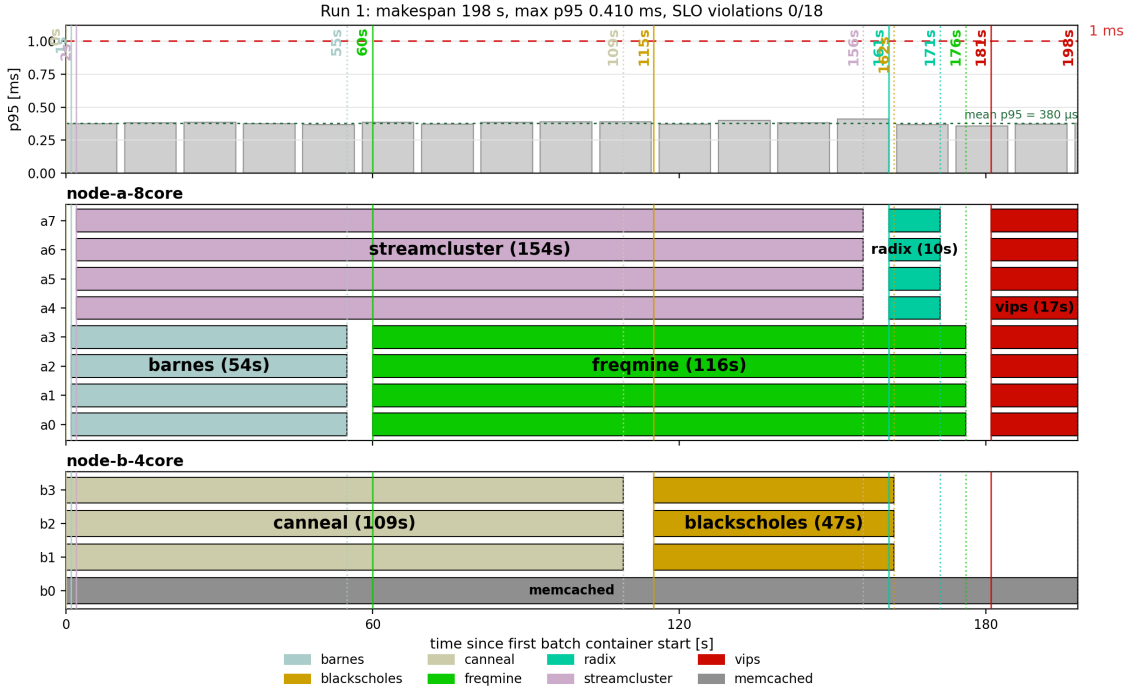
   - Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes	63.33	54.33
blackscholes	47.67	46.67
canneal	107.33	115.00
freqmine	67.33	116.00
radix	20.67	10.33
streamcluster	113.33	155.33
vips	21.33	16.33
total time	232.33	197.33

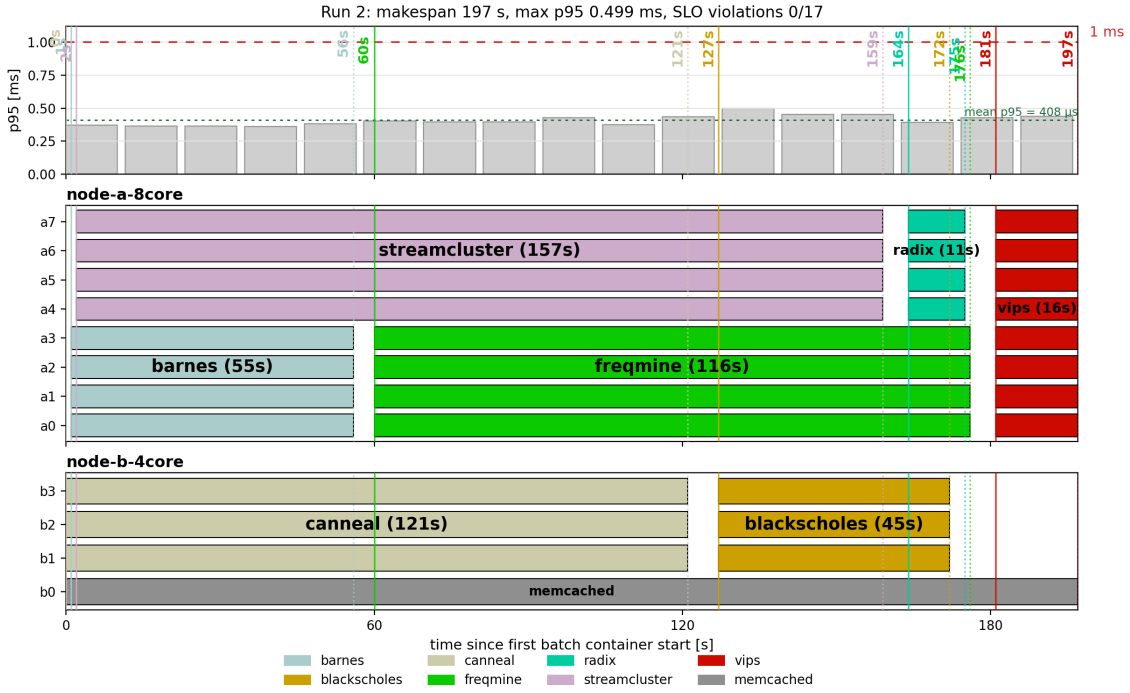
     Answer: The baseline column repeats the mean runtimes from the hand-crafted policy measurements. The AI column is computed from the three final AI-policy executions using completed container runtimes; all seven batch jobs completed successfully in each run. Across the full interval from the first batch container start to the last batch container finish, the memcached p95 latency never exceeded the 1 ms SLO. The SLO violation ratio is therefore $0/63 = 0$.

   - Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.

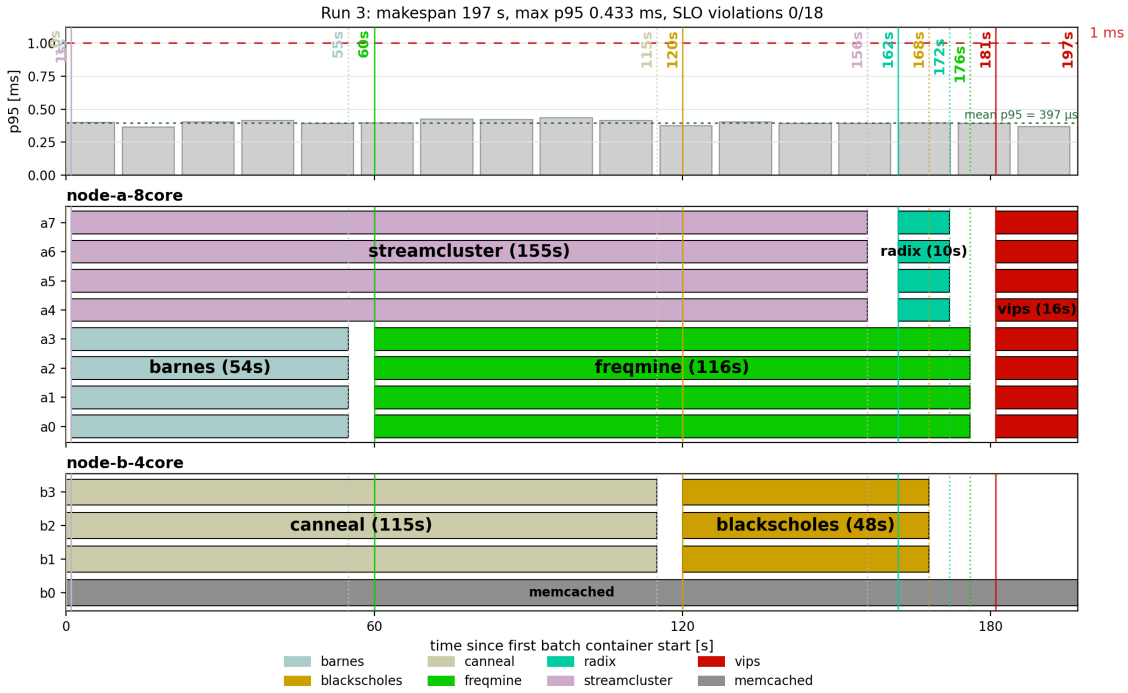
     Answer: The following plots use relative time, with $x = 0$ aligned to the first batch-container start in each run. Solid vertical markers show job starts, dotted vertical markers show job finishes, and the labels on the placement bars include the measured job durations.



Run 1. AI-generated policy: memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.

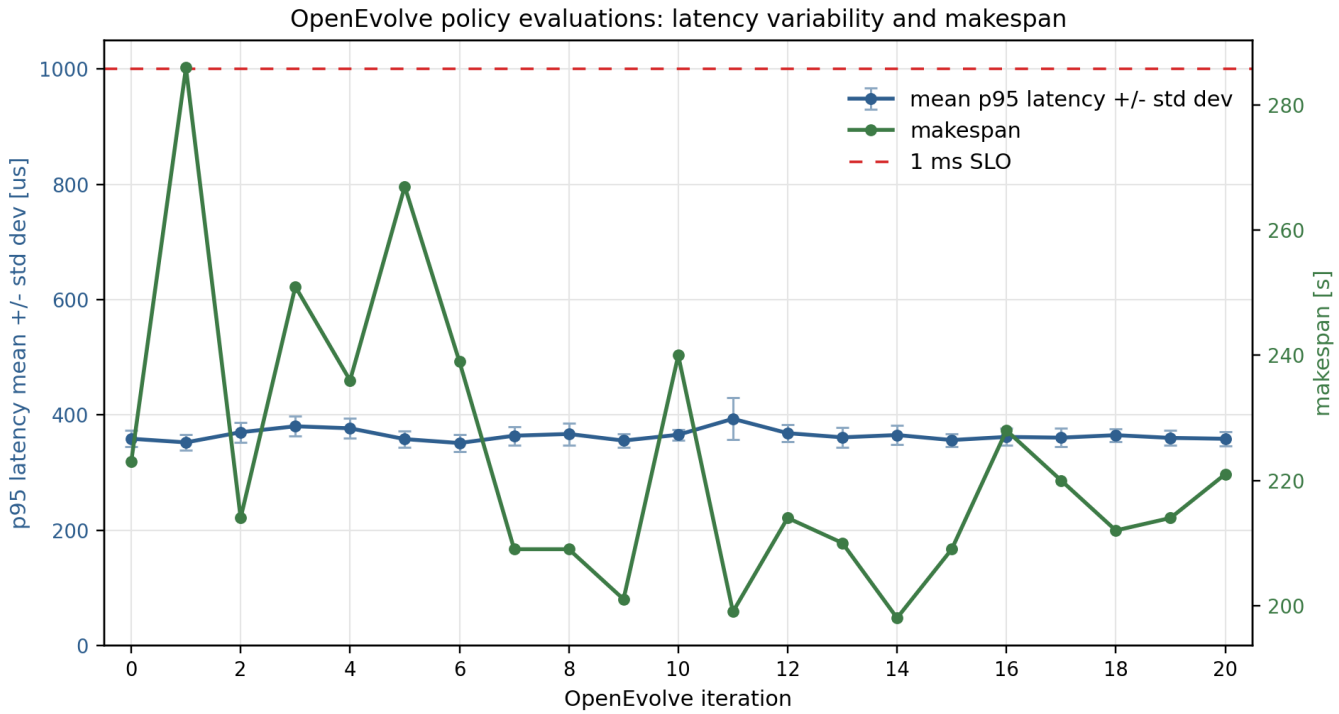


Run 2. AI-generated policy: memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.

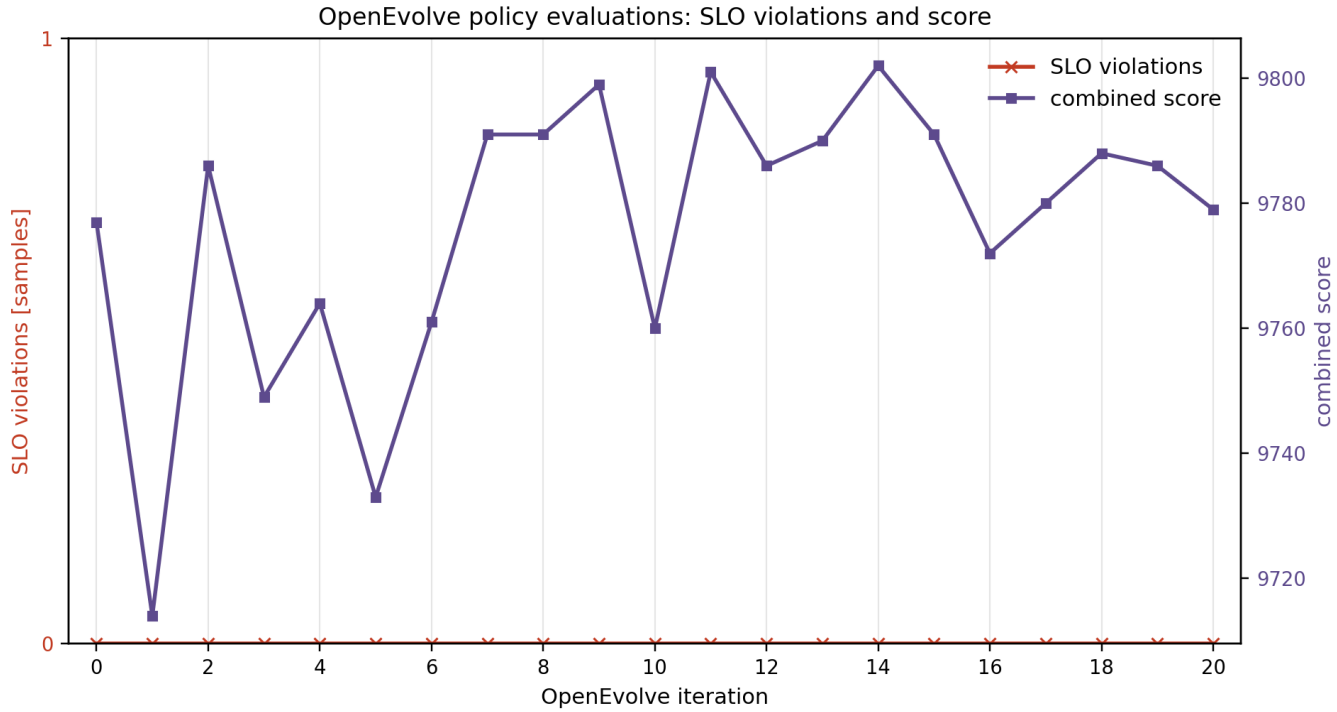


Run 3. AI-generated policy: memcached p95 latency and per-core placement; the red dashed line marks the 1 ms SLO.

- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.
  Answer: We ran OpenEvolve for 20 iterations, starting from the baseline policy at iteration 0. The first plot shows the memcached p95 latency mean with standard-deviation error bars, together with the measured makespan. The second plot shows the SLO violation count together with the evaluator's `combined_score`. For passing policies, the score is $10000 - \text{makespan} - \max(0, \text{max\_p95\_us} - 850)/10$, so in these evaluations it mostly tracks $10000 - \text{makespan}$, because every p95 value stayed well below $850\,\mu$s. Across the saved evaluations, memcached stayed comfortably below the 1 ms p95 SLO and the measured SLO violation count remained zero.



Mean memcached p95 latency with standard-deviation error bars, and makespan, for each saved OpenEvolve policy evaluation. The red dashed line marks the 1 ms SLO.



SLO violation count and `combined_score` for each saved OpenEvolve policy evaluation. Higher score corresponds mainly to lower makespan because none of these candidates incurred a p95 penalty.

Figure 1: OpenEvolve iteration metrics used to analyze the evolutionary process.

(b) **[17 points]** Analysis of the Evolutionary Process: Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the 'combined_score' or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one "hallucination" or logical flaw the model produced during the process and how you identified it.

Answer: Iterations, LLMs, and search setup. We used OpenEvolve as a constrained scheduling-policy search, not as an unconstrained code generator. The benchmarked run used `moonshotai/Kimi-K2.5` and ran through iteration 20, with iteration 0 treated as the seed. We chose Kimi K2.5 because, at the time of our experiment, it was one of the largest reasoning-capable models available on the swiss AI platform, with about one trillion parameters. In the benchmark summaries we considered, it was also ranked among the top 10 open-weight models and among the top 30 frontier models overall. Its benchmark profile suggested strong scientific and agentic reasoning ability, including reported scores of 88% on GPQA Diamond, 39% on GDPval-AA, and 29% on Humanity's Last Exam. This mattered because the scheduling task required the model to reason about a dependency graph, pinned cores, and a latency constraint at the same time. We also tried a later GLM 5.1 experiment, but it was operationally unreliable for this workflow: the model produced much longer thinking traces (up to 50k thinking tokens), OpenEvolve counted those thinking tokens toward the response budget, and the run hit repeated timeout/provider problems. The final policy in the report therefore comes from the Kimi run, which didn't come without it's own problems. In fact the three Kimi log files are also a symptom of the thinking budget timeouts, which we had to address with some restarts and reconfigurations.

A essential part of the setup was the automation harness from the prior exercise, which reduced the LLM's job to emitting a structured policy dictionary. The system prompt told the model that it was acting as a cloud-infrastructure scheduler whose goal was to minimize makespan for seven batch jobs while keeping memcached below a strict 1 ms p95 SLO at 30K QPS. The model could only edit the dictionary inside the OpenEvolve evolve block, and each candidate had to define `policy_name`, `memcached`, and `jobs`. Each job specified `node`, `cores`, `threads`, and `after`; same-node concurrent jobs were not allowed to overlap cores; memcached had to keep a dedicated isolated core; and `radix` was forbidden on the 4-core node. The prompt also encoded empirical guidance from Parts 1–2, for example that `canneal` is cache-sensitive, `blackscholes` is relatively safe

near memcached, and `streamcluster`, `radix`, `barnes`, and `vips` can use the larger node well. The harness then validated the candidate, launched the Kubernetes experiment, collected `mcperf` and job timings, and returned a compact `summary.json` to OpenEvolve.

Success metric. The success metric was implemented in `part3_openEvolve/evaluator.py`. It was designed to make SLO safety a hard requirement, while still allowing OpenEvolve to compare good policies by makespan. For a policy that passed validation and completed the run, the evaluator used the passing score:

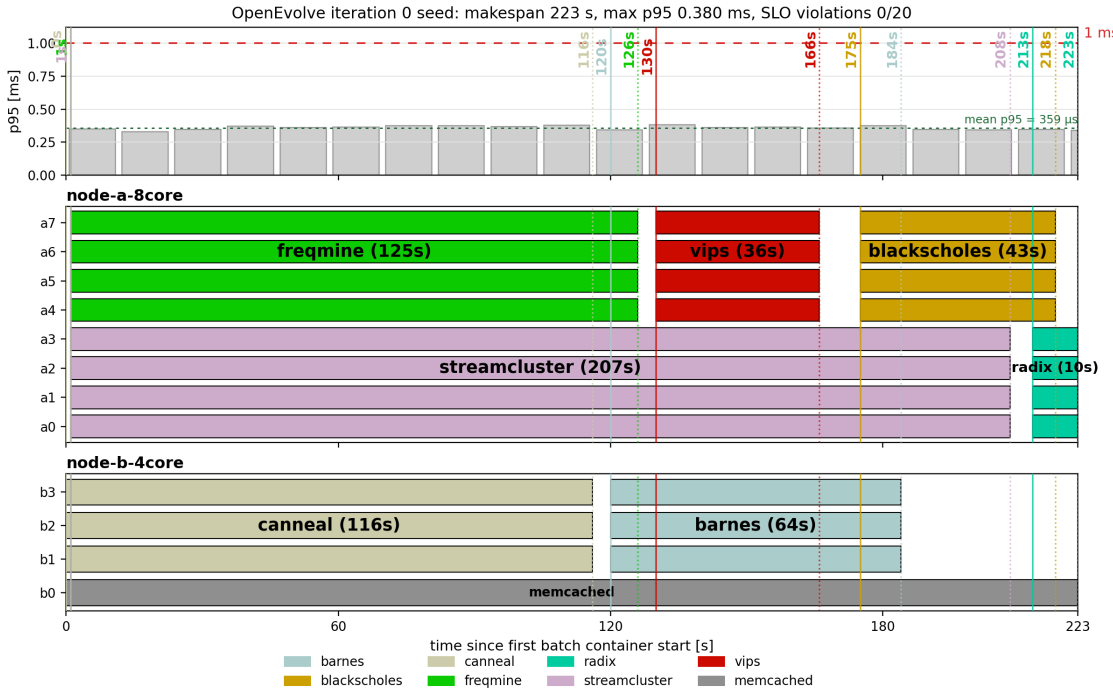
$$\texttt{combined\_score}_{\text{pass}} = 10000 - \text{makespan\_s} - \frac{\max(0, \text{max\_p95\_us} - 850)}{10}.$$

The 850 μs threshold is a safety buffer below the 1 ms SLO. The division by 10 is the chosen scaling: every 10 μs above that buffer costs one score point, so tail latency remains visible in the objective without dominating makespan once the policy is far below the SLO. If a run failed due to SLO violations, the evaluator moved it into a much lower score band:

$$\texttt{combined\_score}_{\text{slo\_fail}} = 1000 - \text{makespan\_s} - 50 \cdot \text{slo\_violations}.$$

Invalid or failed policies received negative scores. In the successful Kimi trajectory, all of the important saved candidates had zero SLO violations and p95 latency well below 1 ms, so the search was effectively optimizing the batch critical path after satisfying the latency constraint.

Initial policy. The initial policy was a handcrafted policy similar to our second-best handcrafted schedule. We used it because it was already legal and latency-safe, but it contained one intentional challenge for OpenEvolve: node B became under-utilized after its initial chain. The seed placed memcached on `node-b-4core` core 0, ran `canneal` → `barnes` on the remaining node-B cores, ran `streamcluster` → `radix` on one 4-core lane of `node-a-8core`, and ran `freqmine` → `vips` → `blackscholes` on the other 4-core lane of node A. This seed scored 9777, with makespan 223 s, max p95 380.2 μs, and zero SLO violations. Figure 2 shows this starting point visually. The tempting fix would have been moving `radix` to node B, but that move was forbidden because `radix` crashed on the 4-core node. OpenEvolve therefore had to find a legal way to shorten the critical path without using that shortcut.



Initial OpenEvolve seed. The schedule is legal and latency-safe, but node B becomes under-utilized after its initial chain; moving `radix` to node B was not a legal fix.

Figure 2: Initial OpenEvolve seed policy.

Evolution of the policy. The evolution was not monotonic, as also visible in Figure 1. The first generated candidate looked more parallel on paper, but it was worse: iteration 1 scored 9714 and increased makespan to 286 s, while still having zero SLO violations. After that, the search recovered. Iteration 2 improved to score 9786 and makespan 214 s; iteration 7 reached 9791 and 209 s; iteration 9 reached 9799 and 201 s; and iteration 11 reached 9801 and 199 s. The best policy was found at iteration 14, with score 9802, makespan 198 s, max p95 398.1 μs, and zero SLO violations. The run continued to iteration 20, but later candidates did not beat this policy.

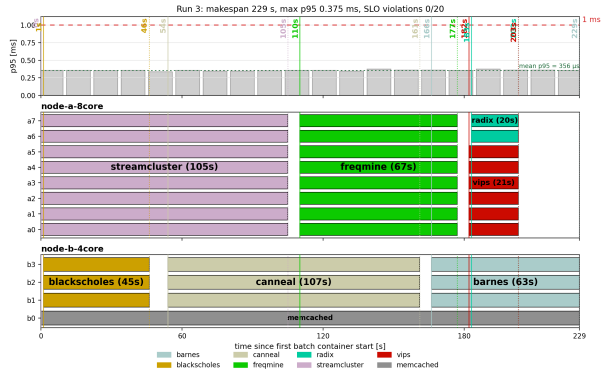
The visualizer helped inspect this process, but we did not treat it as the sole source of provenance. Figure 3 shows both the descendant view and the generation/score view. The red edge in the generation view should be read as the stored immediate `parent_id`, not as the complete set of programs that influenced a new candidate. For example, the best node's stored parent points to a lower-scoring immediate predecessor, but the metadata for the prompt also contained previous/top programs and execution summaries. Therefore, to reconstruct the process, we used the logs and per-program metadata in addition to the visualizer edges.

OpenEvolve descendant view for the Kimi run. The highlighted node is the best-scoring saved policy by `combined_score`.

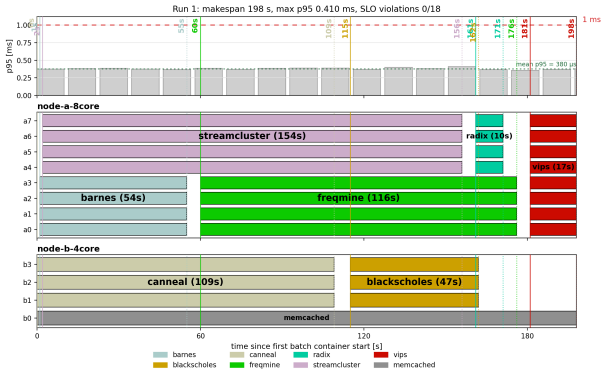
OpenEvolve generation/score view. The red edge shows the stored immediate parent relation, but the policy metadata shows that candidate prompts can also include other previous or top programs as inspiration.

Figure 3: OpenEvolve visualizer views for the Kimi run.

Why the final policy improved. The final AI-generated policy improved makespan by changing the dependency graph rather than by improving tail latency. Our final handcrafted comparison schedule was roughly `streamcluster` → `freqmine` → `max(vips, radix)` on node A and `blackscholes` → `canneal` → `barnes` on node B. In repeated handcrafted runs, the node-B chain alone took about 45–51 s for `blackscholes`, 106–109 s for `canneal`, and 62–65 s for `barnes`, giving a long serial chain of roughly 213–225 s. The OpenEvolve policy removed `barnes` from that node-B chain and moved it to node A. The resulting structure was two node-A lanes, `barnes` → `freqmine` and `streamcluster` → `radix`, while node B ran only `canneal` → `blackscholes`; `vips` then ran after both `freqmine` and `radix`. Some individual node-A jobs became slower because they received fewer cores, but the overall critical path was shorter and better balanced. This is why the best OpenEvolve run reached 198 s, while our repeated handcrafted runs were 229–235 s, with zero SLO violations in both cases. Figure 4 compares the final handcrafted and AI-generated placements side by side.



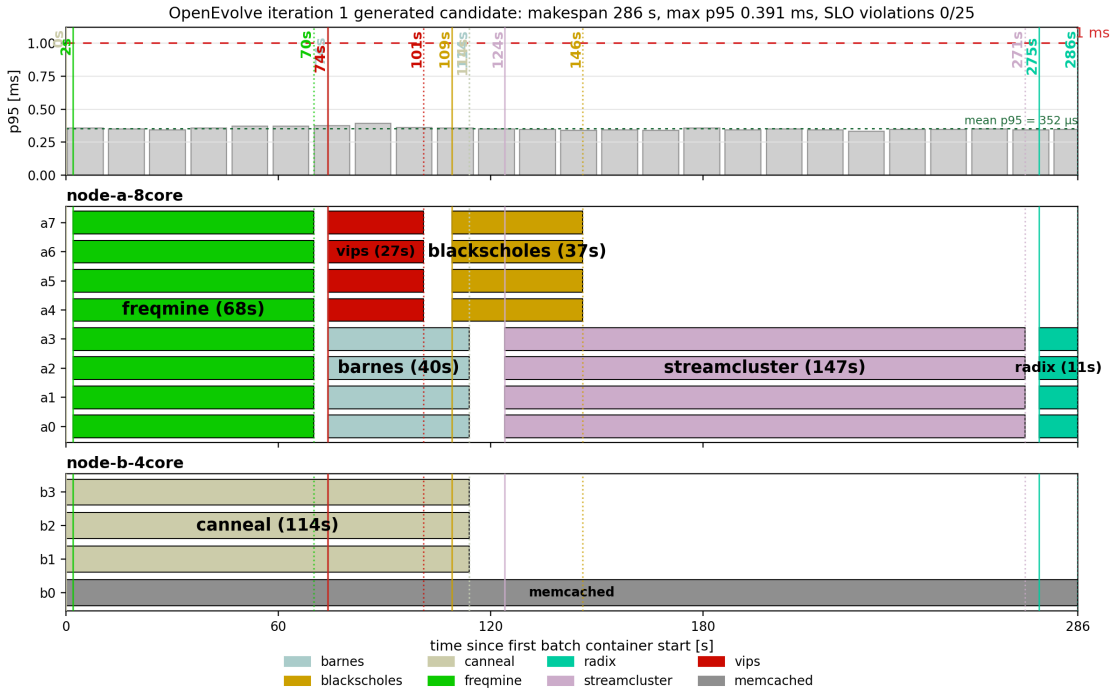
Handcrafted comparison run: makespan 229 s, max p95 375.2 μs, SLO violations 0.



AI-generated policy run: makespan 198 s, max p95 409.9 μs, SLO violations 0.

Figure 4: Side-by-side comparison of the final handcrafted policy and the AI-generated policy.

Logical flaw and how it was identified. One concrete logical flaw appeared in the first generated candidate. It obeyed the schema and tried to parallelize aggressively, but it placed only `canneal` on node B while moving all other batch jobs to node A. This does not necessarily mean the model hallucinated. A plausible explanation is that, before any measured summaries had been fed back to OpenEvolve, the model over-prioritized protecting memcached and therefore avoided colocating more than one batch job with it. We also intentionally did not put detailed job-duration and speedup tables in the system prompt, to avoid overwhelming the model with too much information before the evolutionary loop had produced measurements. The cost of that limited first prompt was visible in the evaluation: compared with the seed, the score dropped from 9777 to 9714 and makespan increased from 223 to 286 s, even though SLO violations stayed at zero. Figure 5 shows this first generated candidate. Subsequent candidates had a better signal because OpenEvolve fed them `summary.json`, a compact version of the Kubernetes timing results from previous evaluations, so later prompts included concrete makespans and job timings instead of only the qualitative scheduling guidance.



Iteration 1 generated candidate. The candidate kept SLO violations at zero, but its node-A chain made makespan much worse: 286 s instead of 223 s for the seed.

Figure 5: Iteration 1 generated candidate with worse makespan despite zero SLO violations.

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.